

AI3002 – Search Methods in AI

Assignment 3: Othello

2025-10-31

In this assignment, you will code a bot to win the game of Othello. Given a board configuration and a turn, your bot will return a valid move. The game ends when neither of the players can make a valid move. The player with a larger number of coins when the game ends is the winner.

Programming Language: C++

System Specification (preferred): Linux or WSL (Use MinGW only if you're an expert)

1. Instructions

1.1. Setting up the Framework

We will be providing you with a framework ([Desdemona](#)) that lets two bots compete against each other.

1. Extract the contents of Desdemona into a suitable directory.
2. Set up the framework by issuing a make command in the root of this directory.

1.2. Coding the Bot

- You will modify MyBot.cpp to return a valid move whenever the function playis called. The source is located at bots/MyBot.
- All other source files are to be left untouched.
- The makefile is also provided at this location. Use it to generate a .so file.
- You can test your bot against another bot by issuing the command

```
1 ./bin/Desdemona ./<path to bot1.so> ./<path to bot2.so>
```

- By convention, the first bot is BLACK and the second RED.
- A random bot (bots/RandomBot) has been provided for testing.

The following is an example -

```
1 ./bin/Desdemona ./bots/RandomBot/RandomBot.so ./bots/RandomBot/RandomBot.so
```

- At the end of the game, a game.log file is created that contains the sequence of moves made.
- The bots being submitted must have NO print statements.
- If a bot returns an invalid move, it will be disqualified.

1.3. Helper Functions

The following functions have already been written to assist you:

1.3.1. Validate Move

```
1 bool OthelloBoard::validateMove( Turn turn, int x, int y )
```

true if the passed move (x,y) is valid for the passed turn, false otherwise

```
1 bool OthelloBoard::validateMove( Turn turn, Move move )
```

true if the passed move is valid for the passed turn, false otherwise

1.3.2. Make Move

```
1 void OthelloBoard::makeMove( Turn turn, int x, int y )
```

Updates the board configuration by making the move (x,y); throws an exception if the move is not valid

```
1 void OthelloBoard::makeMove( Turn turn, Move move )
```

Updates the board configuration by making the specified move; throws an exception if the move is not valid

1.3.3. Get Valid Moves

```
1 list<Move> OthelloBoard::getValidMoves( Turn turn )
```

Returns a list of valid moves that can be made given the turn

1.3.4. Get Black Count

```
1 int OthelloBoard::getBlackCount()
```

Returns the number of black coins on the board

1.3.5. Get Red Count

```
1 int OthelloBoard::getRedCount()
```

Returns the number of red coins on the board

1.3.6. Print Board

```
1 void OthelloBoard::print( Turn turn )
```

Prints the turn, the board configuration, and the number of black and red coins. 'X' is BLACK, 'O' is RED, and unfilled locations are blank

1.4. Time Constraints

Each bot can take at most 2 seconds to return a move. If this time limit is exceeded, the bot causing the timeout will be disqualified.

1.5. Evaluation

- The assignment is to be done in teams of three.
- A complete round robin tournament of all bots will be run. The outcome will determine the marks for the assignment.
- If running a complete tournament is not possible then each bot will be pitted against 5 other randomly chosen bots and the results will be translated to assignment marks.

2. Submission Details (7%)

2.1. Trial Submission (1%)

Upload a single .so file with your IDs (U202300XX_U202300YY_U202300ZZ.so). You get full if your bot works without going out of the time constraint.

2.2. Final Submission (6%)

Upload a zip file containing your source code, writeup and the .so file. The zip file is to be named as U202300XX_U202300YY_U202300ZZ.zip. Write should be a well-documented README.md outlining your approach.